Fast Document Retrieval Engine for a University RAG Assistant

PBL Technical Report: CS-251 Data Structures and Algorithms (25CS)
Department of Computer Systems Engineering, Mehran University of Engineering and Technology, Jamshoro
Team: Waqas Ali (25CS061) · Mehran Ali (25CS009) · Gaffar (25CS067) · Basit Azam (25CS113)
Subject teacher: Dr. Ali Asghar Manjotho · **Submitted:** [date]

All figures in this report come from `rag eval` and `rag test`, run on one laptop (Windows 11, g++ 10.3 `-O2`). Raw output is in `results/`. Timings vary slightly between runs; the trends and ratios are stable.

Data notice: the corpus is written in the context of MUET's Computer Systems Engineering department. Apart from document D24, which is taken from the PBL task brief, **every rule is synthetic** and is not official MUET policy (brief §10).

1. Problem framing

The department wants a question-answering assistant for course outlines, lab rules, exam regulations and notices, using retrieval-augmented generation (RAG). Our job was the **retrieval** step: given a question, return a small set of passages the assistant can cite. The current approach scans every passage for every question, which gets slower as the collection grows and can surface outdated notices.

Definitions we agreed on (Week 1)

Term	Our definition
Document	One source file (e.g. "Attendance Policy Notice 2026") with an id, title, category, version, date and status
Passage	One self-contained statement from a document (1–2 sentences), the unit we retrieve and cite
Query	Free text from a user; turned into a set of *concepts* (distinct terms + optional synonyms)
Relevant result	A passage a human labeller judged to contain the answer or a necessary part of it (gold label)
Search latency	Wall-clock time of one `search()` call, median of repeated runs, measured separately from index build
Traceable evidence	Each returned passage carries passage id, document id/title, version, date, status, an excerpt and a score explanation

Success measures:

- correct results (Recall@3 on labelled questions);
- correct refusals ("No supported evidence found" on unanswerable questions);
- query latency compared with the baseline;
- index memory overhead;
- current information presented ahead of superseded notices.

Assumptions:

- The corpus fits in memory.
- Passages are short and already split.
- English text, ASCII-tokenisable.
- A superseded document stays searchable but must be flagged.
- An LLM answer is out of scope.

2. Inquiry: learning questions and alternatives

Learning questions recorded in Week 1, and what we found:

1. *How should hash collisions be handled, and how bad can it get?* We chose separate chaining: it is simple, deletion is easy, and it degrades gracefully. Test T11 forces every term into one chain (670 terms in one list) and the index still returns identical answers, just slowly.
2. *What does an inverted index have to store to rank, not just match?* Term frequency per passage (for BM25) and document frequency, which is simply the length of the postings list.
3. *How do we rank so an old notice does not beat its replacement?* We added a transparent freshness rule (superseded passages × 0.5) and compared it with a ranker that has no such rule.
4. *How is Recall@k computed fairly when duplicates are folded and subsets drop gold passages?* See §7.2.
5. *When does a heap beat sorting for top-k?* When the number of candidates C is much larger than k (see §8).

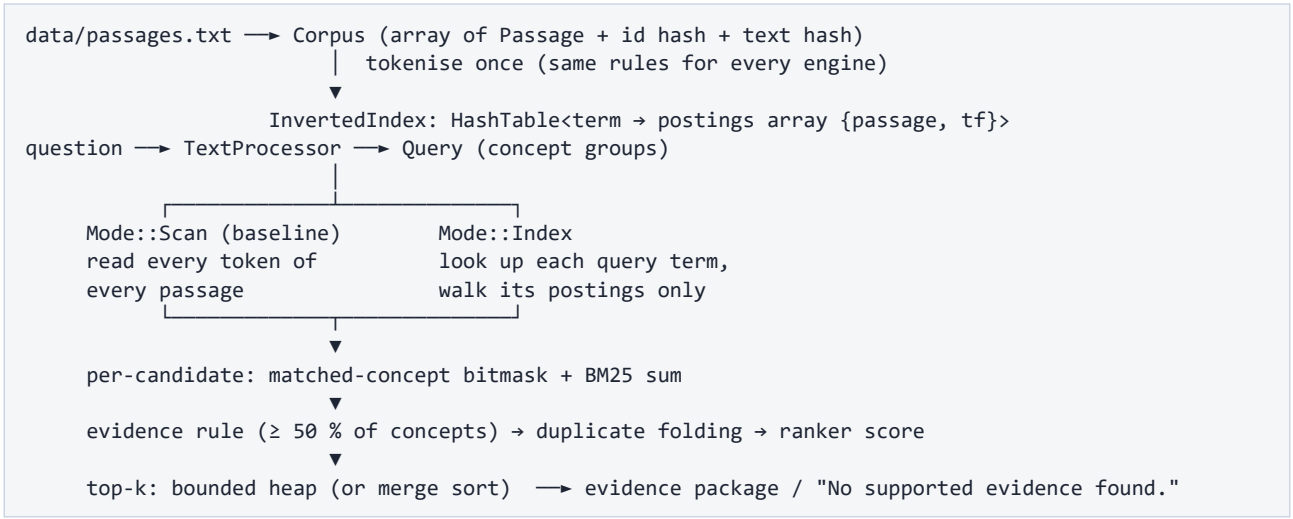
Alternatives considered:

Area	Options investigated	Chosen	Why
Retrieval architecture	(a) linear scan of stored passages; (b) inverted index over a hash table; (c) sorted vocabulary array + binary search	(a) as baseline, (b) as engine	(c) costs O(log V) per lookup and makes insertion O(V); a hash table gives expected O(1) lookup and cheap additions
Collision handling	separate chaining vs open addressing (linear probing)	chaining	Deletion is simple (needed for updates); the table never "fills up"; load factor above 1 still works
Postings	linked list of nodes vs growable array	growable array `{passage, tf}`	Cache-friendly sequential traversal; append is amortised O(1); removal is rare
Ranking	shared-term count; normalised coverage; TF-IDF; BM25; BM25 + freshness	coverage (policy A) and BM25 + freshness (policy B)	A is fully explainable; B weights rare terms and handles versions
Top-k	sort all candidates (merge sort); bounded heap	bounded heap (merge sort kept for comparison)	O(C log k) vs O(C log C); identical results (T16)

What changed in our design during the work:

- Exact duplicates initially produced the same text twice in the top 3, so we added the duplicate-text hash and folding.
- The paraphrase questions scored 0, so we added a small, declared synonym list and report results both with and without it.
- The 2025 attendance notice ranked first under coverage scoring, which led to the freshness rule.

3. System architecture



There are two front ends:

- the command-line program `rag`;
- a desktop GUI (`gui/rag_gui.py`, tkinter). It runs `rag --json` and only displays the results; all retrieval logic stays in C++.

Data structures used, and where:

#	Structure	Built by	Where / purpose
1	**Hash table, separate chaining** (FNV-1a, singly linked chains, doubles at load 0.75)	**us** (`hash_table.h`)	vocabulary → postings; passage-id → index; normalised-text → first passage (duplicates); stop-word set; synonym map
2	**Bounded binary heap** (array-based; root = worst kept)	**us** (`top_k_heap.h`)	top-k selection, O(C log k)
3	**Merge sort** (stable, top-down)	**us** (`merge_sort.h`)	alternative top-k policy, O(C log C)
4	Arrays	std::vector used as a buffer	passage store, postings lists, per-query scratch arrays indexed by passage
5	Linked lists	**us** (hash chains)	collision chains inside the hash table
6	Bitmask per candidate	us	which query concepts a passage matched (coverage)

Preprocessing (`text.cpp`, identical for passages, queries, baseline and index):

- split on non-alphanumeric characters, so `CS-251` becomes `cs` + `251`;
- lower-case every token;
- remove 77 stop words;
- normalise suffixes: `-ies` → `-y` and trailing `-s` dropped;
- collapse repeated query terms into one concept (T04);
- a blank or stop-word-only query is reported as empty (T02).

Evidence rule and output. A passage counts as evidence only if it matches at least half of the query's concepts. This stops "parking permit for student cars" from returning passages that merely mention *student* (46 weak candidates, 0 evidence; T18). Each result prints:

- its id, document and version/date;
- a **CURRENT / SUPERSEDED** status, with a warning on superseded text;
- an excerpt;
- which concepts matched and which were missing;

- the BM25 value and freshness factor;
- any duplicate copies.

Safe handling. Passage text is printed only as quoted data; it is never parsed for commands. Passage D22-P3 contains a planted "IGNORE ALL PREVIOUS INSTRUCTIONS …" sentence. It is retrieved like any other passage and changes nothing (T19).

Additions and updates. New passages are appended to the store and their postings appended to each term's list. An update removes the old passage's postings, marks its slot deleted and indexes the new text (T20).

4. Ranking investigation

Both policies run on **exactly the same candidate set** (same evidence rule):

- **Policy A, Coverage:** `score = matched concepts / total concepts`. It is easy to explain, but it ignores how rare a term is and ignores versions.
- **Policy B, BM25 + freshness:**
  - Per concept, take the best BM25 weight among its terms: `idf · tf(k1+1) / (tf + k1(1 − b + b·len/avglen))`, with k1 = 1.2, b = 0.75 and `idf = ln(1 + (N − df + 0.5)/(df + 0.5))`.
  - Sum over concepts.
  - Multiply by 0.5 if the passage is superseded.
- **Deterministic tie-break, both policies:**
  1. higher score;
  2. current before superseded;
  3. newer document date;
  4. earlier position in the file.

What we observed:

- Both policies reach the same Recall@3 (0.912 on 155 passages), because with k = 3 the gold passages fall inside the top 3 either way.
- The **order** differs where it matters:
  - **Q09 (attendance):** Coverage ranks the superseded 2025 passage D04-P3 first (4/5 concepts). BM25 + freshness ranks the current D09-P1 first (T12, T14).
  - **Q10 and Q18 (date sheet, MUET registration notice):** the revised notice (D20-P2, D29-P1) is ranked first and the original is marked SUPERSEDED (T13, T13b).
  - **Q11 (marks):** BM25 puts the shorter passage D06-P4 above D01-P3 (length normalisation). Both are gold.
- **Selected:** policy B as the default, because it presents current rules first. Policy A is available via `--ranker coverage`.

5. Dataset and evaluation questions

Full details are in `data/DATASET.md`.

- **Corpus:** 31 documents, 155 passages, 5 topics (course, lab, exam, notice, admissions).
  - D01–D23: general academic documents (synthetic).
  - D24: facts from the CS-251 PBL task brief (real).
  - D25–D31: MUET CSE-context notices (synthetic): lab timetable for batch 25CS, FYP guidelines, internship policy, original and revised registration notices, department office, viva guidelines.
  - Includes three old/new pairs, exact and near duplicates, and one planted injection passage.
- **Questions (20):**

Type	Count
Keyword	8
Paraphrase	4
Ambiguous / version	5
Unanswerable	3

- Every relevant passage was labelled by hand. Gold sets contain only current passages.
- **Corpus sizes:** the first 25, 50 and 100 passages, plus all 155. Documents are interleaved by topic so every subset covers at least 4 topics.

6. Testing and validation (`rag test`: 24/24 pass)

Id	Case (brief §10 minimum cases in bold)	Result
T01	**Empty corpus**, both engines	"No supported evidence"; reason: corpus empty
T02	**Blank query** / stop-words only	Reported as empty after preprocessing
T03	**Term not present**	0 candidates, no evidence
T04	**Repeated query terms**	Collapse to 1 concept; identical ranking
T05	Repeated term inside a passage	1 posting, tf = 2
T06	**Multiple matching passages**	15 candidates, k = 3 returned, scores descending
T07	**Exact duplicate passages**	D02-P1 shown once; D23-P1 folded
T08	Near-duplicate	Not detected (documented limitation)
T09	**Hash collisions**: 4 buckets, 200 keys, no resize	All found; 196 colliding entries
T10	Worst-case hash (all keys in 1 bucket)	find / erase / insert correct
T11	Whole index on a degenerate hash (670 terms in one chain)	Same answers on all 20 questions
T12, T13, T13b	**Old/new versions** (attendance, date sheet, MUET registration)	Current notice first; superseded flagged
T14	Coverage ranker on a version query	Shows the superseded-first failure that motivated policy B
T15	Scan vs index	40/40 identical ids **and** scores
T16	Heap vs merge sort	60/60 identical top-k
T17	k = 50, k = 0	Only qualifying passages returned; k = 0 handled
T18	Unanswerable questions	No evidence, no invented rule
T19	Injection text in a passage	Returned as data only
T20	Add / update / remove after indexing	Correct at each step; postings and vocabulary cleaned
T21	Duplicate passage_id	Rejected at ingestion
T22	Tie-break	Deterministic
T23	Heap and merge-sort unit check	Correct

The expected and observed text for each case is in `results/edge_case_tests.txt`.

7. Performance and retrieval-quality results

7.1 Method

- Same machine, same query set, k = 3, same ranker settings for both engines.

- Each query: 5 warm-up runs, then 15 measured runs. Each run times 40 back-to-back searches and divides, because a single ~1 µs search is close to the clock's resolution. The **median** of the 15 runs is reported.
- Index build time is measured separately (median of 7 builds).

7.2 Recall@3

`Recall@k = gold passages among the top k / all gold passages`, averaged over answerable questions.

- A gold passage also counts if its exact duplicate was returned, because duplicates are folded by design.
- On the smaller corpora, only gold passages present in that subset are counted. Questions with no gold passage in the subset are timing-only.

Corpus	Answerable questions scored	Recall@3: coverage	Recall@3: BM25 + fresh	Unanswerable refused
25 passages	6	0.833	0.833	3/3
50 passages	10	0.800	0.800	3/3
100 passages	12	0.875	0.875	3/3
155 passages	17	**0.912**	**0.912**	**3/3**
155, synonyms OFF	17	0.794	0.794	3/3

Per-question view (155 passages, BM25 + fresh):

- **Q01–Q06, Q08, Q10, Q11 and Q15–Q19** = 1.00.
- **Q07 = 0.50:** "cheating in a paper" finds D08-P2 but not D08-P3 ("repeated cases of cheating may lead to expulsion"). D08-P3 matches only 1 of 4 concepts, so the evidence rule drops it.
- **Q09 = 0.50:** D09-P3 ("students below 75 percent attendance are detained") shares only *attendance* with the question.
- **Q12 = 0.50:** D01-P4 says "late assignments lose 10 percent", which has no "submission" or "penalty". The MUET late-registration fine (D29-P3) now also enters the top 3; it shares *late* and *fine* but is not a submission penalty.

Synonym ablation. Turning expansion off drops Q06 and Q08 to 0. Our synonym list was written while looking at the corpus, so 0.912 is optimistic for paraphrase questions; 0.794 is the keyword-only figure.

7.3 Query latency: baseline vs index

Median of per-query medians, BM25 + freshness:

Passages (N)	Tokens (T)	Scan	Index	Speed-up
25	284	3.3 µs	0.57 µs	5.8×
50	524	5.6 µs	0.88 µs	6.3×
100	986	11.2 µs	1.35 µs	8.3×
155	1,557	17.7 µs	1.48 µs	11.9×

Scaling run (runtime only). The corpus is replicated 10×, 50× and 100×. Each copy gets a unique marker word so copies are not folded as duplicates. Relevance labels do not apply to this run.

Passages	Build	Vocabulary V	Postings	Index memory	Scan	Index	Speed-up
155	0.16 ms	670	1,501	64.0 KB	16.7 µs	1.5 µs	11.2×
1,550	1.0 ms	679	16,405	238.7 KB	186 µs	14.8 µs	12.6×
7,750	3.8 ms	719	82,645	953.9 KB	1,012 µs	64.1 µs	15.8×
15,500	7.2 ms	769	165,445	1,868 KB	2,257 µs	126 µs	17.9×

Interpretation:

- **The scan grows with T**, the total number of tokens.
- **The index grows with P**, the postings of the query terms only, so its time grows much more slowly, and the speed-up rises from about 6× to about 18× as N grows.
- **A real collection would do better.** Replication repeats the same vocabulary, which inflates P; with real, diverse documents the gap would widen further.
- **Outliers:** the slowest index queries (Q10, Q11, Q17, about 3 µs) use common terms such as *final*, *examination*, *student* and *project*, which have the longest postings lists.
- **Noise:** single-run timings vary by roughly ±10 % (background processes, CPU frequency); medians over repeated runs remove most of it.

7.4 Index memory

Accounting method:

- bucket array = number of buckets × pointer size;
- nodes = entries × sizeof(node), where a node holds the key string header, the postings vector header and the next pointer;
- long-key heap storage = keys longer than the 15-byte short-string buffer;
- postings = list capacity × 8 bytes per `{int, int}` posting.

Document text and token arrays are reported separately; they are needed by both engines and are not index overhead.

Corpus	Buckets	Nodes	Postings	Index total	Document text + metadata	Token arrays
25	4.0 KB	12.3 KB	2.3 KB	**18.6 KB**	12.6 KB	12.5 KB
50	4.0 KB	19.4 KB	4.4 KB	**27.8 KB**	24.9 KB	23.0 KB
100	8.0 KB	30.7 KB	8.5 KB	**47.2 KB**	49.0 KB	43.5 KB
155	8.0 KB	41.9 KB	14.1 KB	**64.0 KB**	93.1 KB	68.2 KB

On this small corpus most of the index is per-term node overhead (V = 670 nodes × 64 B). With replication, postings dominate and the index grows to about 1.9 MB for 15,500 passages.

Hash table health at 155 passages:

Measure	Value
Buckets	1,024
Load factor	0.65
Resizes	4
Longest chain	4

8. Algorithmic complexity

Notation:

Symbol	Meaning
N	passages
T	total tokens
V	unique terms
q	expanded query terms (small; ≤ 64 concepts)
P	postings visited for the query's terms
C	candidates (C ≤ min(N, P))

Symbol	Meaning
k	results requested

Operation	Time	Extra space	Notes
Preprocess corpus	O(total characters)	O(T)	done once at load
Index build	O(T) expected; O(T·V) worst case	O(V + T)	hash insert expected O(1); doubling rehash is amortised O(1) per insert
Add one passage	O(t_p) expected	O(t_p)	t_p = tokens in that passage
Remove / update passage	O(Σ df(t) for its terms)	O(1)	array erase keeps postings ordered
Sequential retrieval	**O(T·q)**	O(C·q)	reads every token of every passage
Indexed lookup	O(q) expected; O(q·V) worst case	—	worst case = all terms in one chain (T11)
Candidate scoring (index)	O(P)	O(N) scratch, allocated once, reset only for touched entries	
Evidence rule + folding	O(C)	O(C)	
Top-k, bounded heap	**O(C log k)**	**O(k)**	
Top-k, merge sort	O(C log C)	O(C)	

Why the heap wins. Measured on 15,500 passages with a broad query (C = 9,700 candidates):

k	Heap	Merge sort
3	0.71 ms	2.98 ms
100	0.80 ms	2.91 ms

Both return identical top-k lists. Sorting everything only pays off when k approaches C, or when the full ranked list is needed anyway (e.g. for pagination).

Expected vs worst-case hashing. FNV-1a spreads our vocabulary well (longest chain 4). A pathological hash degrades lookups to a linear chain walk. T11 shows the answers stay correct even then; only speed suffers.

9. Limitations

- **Keyword matching misses paraphrases** (Q06 and Q08 fail without synonyms). The synonym list is small and was written with knowledge of the corpus.
- **The suffix rules are crude** ("classes" → "classe", "25CS" → "25c"). They are consistent, but they don't merge "submit" with "submission".
- **Only exact or normalised duplicates are folded;** near-duplicates are not (T08).
- **The evidence threshold (50 %) is a fixed heuristic.** It rejects some true evidence (Q07, Q09, Q12) and was tuned on this small question set.
- **Version status is manual metadata.** The engine cannot work out on its own that one notice replaces another.
- **Speed-up figures beyond 155 passages come from a replicated corpus.**
- **The MUET-context documents are synthetic.** A real deployment would ingest the department's actual notices.

10. Reflection and next steps

Predictions vs observations:

- We predicted the index would win by a large factor. It does, and the gap grows with corpus size (about 6× at 25 passages, about 12× at 155, about 18× at 15,500). On tiny corpora the absolute difference is under 3 µs.
- We predicted BM25 would beat coverage on recall. On this set, recall is equal; the real benefit was **ordering** (current notices first).
- The largest quality gains came from data handling (duplicates, versions, the evidence rule), not from the scoring formula.

Next steps:

1. Phrase or proximity scoring.
2. A larger synonym resource.
3. Near-duplicate detection.
4. Automatic version linking via a `replaces` field.
5. Compressed postings for larger collections.
6. Optional hand-off of the evidence package to an LLM. This must be evaluated separately from retrieval, because Recall@3 measures retrieval only, not answer quality.

Appendix A: How to reproduce

```
build.bat
rag test
rag eval
run_gui.bat
```

Outputs: `results/edge_case_tests.txt`, `results/evaluation_summary.txt`, `results/query_results.csv`, `results/latency.csv`, `results/scaling.csv`.

Appendix B: Individual contributions

See `docs/PROJECT_MANAGEMENT.md`: section 1 (work split, about 25 % each) and section 7 (contribution notes).